

Lab 08 – Passive OT IDS with Zeek – Solutions

Industrial Security (Bachelor)

• Maps to Section 08

• 45–60 min

LAB 08

🎯 Goal

Lecturer / TA reference: expected log entries, sample patched policy, SOC escalation answer, grading rubric.

✅ Step 1 – Baseline polls and log inspection

After one minute of `modbus_poll_loop.py`, `ls zeek-logs/` on the host should list roughly:

```
1 conn.log modbus.log known_modbus.log packet_filter.log reporter.log
```

`dns.log` usually appears too if the student container resolved anything during start-up. A representative `tail zeek-logs/modbus.log` line (JSON, one record per poll):

```
1 {"ts":1747500123.456,"uid":"CXyZ12abc","id.orig_h":"172.28.0.20",
2  "id.orig_p":54321,"id.resp_h":"172.28.0.10","id.resp_p":502,
3  "tid":1,"unit":1,"func":"READ_HOLDING_REGISTERS"}
```

Teaching point: function code 3 (Read Holding Registers) is a benign poll. Source 172.28.0.20 is the student / engineering workstation, destination 172.28.0.10 is OpenPLC on TCP/502.

✅ Step 2 – Trigger the injection

`modbus_inject.py` issues a Write Single Coil (FC 5) at address 0 with value 0xFF00. Within a second or two, `tail zeek-logs/notice.log` shows a JSON record like:

```
1 {"ts":1747500188.012,"note":"ModbusWatch::Unsolicited_Write",
2  "msg":"Modbus write from 172.28.0.20 to 172.28.0.10 (FC=5)",
3  "src":"172.28.0.20","dst":"172.28.0.10","p":502,
4  "uid":"CabCDe456"}
```

`modbus.log` continues to receive both the poll requests *and* a new entry with `func="WRITE_SINGLE_COIL"`. The analyzer always logs what it parses; the NOTICE stream is the alert-worthy subset.

✅ Step 3 – Tune one false positive

Edit `bachelor/labs/_stack/zeek/local.zeek` on the *host*, then `docker compose restart zeek`. The patched event handler short-circuits when the source is the sanctioned engineering workstation:

```
1 event modbus_message(c: connection, headers: ModbusHeaders,
2                      is_orig: bool) {
3     if (!is_orig) return;
4     # Sanctioned engineering workstation -- writes expected.
5     if (c$id$orig_h == 172.28.0.20) return;
6     if (headers$function_code in set(5, 6, 15, 16)) {
7         NOTICE([$note=Unsolicited_Write,
8                 $msg=fmt("Modbus write from %s to %s (FC=%d)",
9                           c$id$orig_h, c$id$resp_h,
10                          headers$function_code),
11                 $conn=c]);
12     }
13 }
```

After the restart, re-run `modbus_inject.py`. Expected outcome:

- `notice.log` receives *no* new `ModbusWatch::Unsolicited_Write` entry.

- `modbus.log` *does* still receive a fresh `WRITE_SINGLE_COIL` record – the analyzer still parses the PDU, only the NOTICE is suppressed.

✓ Model 5-line answer – SOC escalation

“Who would receive this alert in a real SOC, and what is their first action?”

1. A tier-1 SOC analyst on the OT queue (or the on-call OT IR engineer at smaller sites with no dedicated OT shift).
2. First action: look up the source IP in the asset inventory and confirm it is *not* a sanctioned engineering workstation.
3. Cross-check with the on-shift operator / control room: is a commissioning, recipe change, or maintenance window in progress that legitimately writes to FC 5/6/15/16?
4. If unsanctioned, escalate to OT IR and the plant manager *before* any containment action.
5. Never unilaterally pull the network cable or block the source – in OT, an unannounced isolation can trip a safety instrumented system (SIS) and cause an unsafe shutdown.

✓ Common student mistakes

- Editing `local.zeek` *inside* the running container (`docker compose exec zeek vi ...`). The bind mount goes the other way – changes inside the container vanish on the next restart. Always edit on the host under `_stack/zeek/`.
- Forgetting `docker compose restart zeek` after the edit. Zeek loads the policy once at start-up.
- Suppressing the wrong source. `172.28.0.10` is the PLC itself; writing `if (c$id$orig_h == 172.28.0.10) return;` silences responses that the analyzer would not raise anyway, while leaving the actual attacker IP loud.
- Expecting `modbus.log` to also fall silent after tuning. It should not – the tuning only suppresses the NOTICE, the Modbus analyzer keeps logging every PDU.
- Confusing function codes: FC 5 = Write Single Coil (the inject script), FC 6 = Write Single Register, FC 15/16 = the multi-write variants. All four belong in the watchlist.

Grading rubric

Full marks (100 %) require:

- `notice.log` excerpt from Step 2 containing exactly one `ModbusWatch::Unsolicited_Write` for FC 5 from `172.28.0.20` to `172.28.0.10`.
- Modified `local.zeek` that suppresses writes from the sanctioned source *and* still raises the NOTICE for any other source (i.e. the early-return is guarded by the source IP, not by removing the FC check).
- Post-tuning `notice.log` excerpt showing no new entry after re-running `modbus_inject.py`, together with the matching `modbus.log` line that proves the write was still observed.
- 5-line SOC answer naming the right responder (tier-1 OT SOC / on-call OT IR), the asset-inventory + operator check as the first action, and the explicit warning against unilateral containment in OT.

Partial credit: –20 % for an over-broad suppression (e.g. removing the FC set entirely, or matching on `id$resp_h` so all writes to the PLC are silenced regardless of source); –15 % for missing the pre/post `notice.log` comparison; –15 % for a SOC answer that jumps straight to network isolation without the asset / operator verification step.